

Trabalho Prático 0

Disciplina Estrutura de Dados – Operações com Matrizes

Raphaela Silva

¹Departamento de Ciência da Computação – Universidade Federal de Minas Gerais

Belo Horizonte – MG - Brasil

rapha470@gmail.br

1. Introdução

A proposta do TP foi implementar um programa que realiza operações com matrizes bidimensionais (soma, multiplicação e transposição), alocando-as dinamicamente na memória. As matrizes de entrada devem ser disponibilizadas em arquivo e a matriz de saída também, assim como o tamanho das matrizes.

O programa também deve realizar a avaliação de desempenho do código usando a biblioteca ‘memlog’. Os erros do programa devem ser tratados com as macros definidas em ‘msgassert.h’.

2. Método

O programa foi implementado usando um vetor de ponteiros de linhas contíguas. Foi preferível essa estrutura de dados àquele vetor de ponteiros de linhas separadas ou àquele vetor de alocação única por causa da facilidade que o primeiro tem em operações de leitura e escrita de arquivos. Essa facilidade se dá pelo fato de todos os elementos da matriz serem alocados em sequência na memória.

Essa estrutura de dados foi montada em um struct, tendo como membros um vetor, número de linhas e colunas da matriz. Esse struct tem as funções para criar a matriz, inicializar uma matriz nula, escrever a matriz em um arquivo, somar, multiplicar e transpor matrizes e destruir a matriz.

No programa principal é lida a entrada do usuário e para cada opção de operação é executado uma série de funções específicas. Além do código principal também executar as funções para ativar e finalizar o registro de memória.

Na função para criar a matriz é lido do arquivo que contem a matriz o número de linhas e colunas e então é inicializada uma matriz nula a partir de uma função. Só então o arquivo é transcrito para a matriz que uma vez foi nula.

Na função de inicializar a matriz nula primeiro é alocado um vetor de ponteiros para linhas e então é alocado um vetor em que cabe todos os elementos da matriz. Sequencialmente, os ponteiros para as linhas são ajustados para a matriz, então a matriz é percorrida assinalando zeros em cada elemento da matriz.

A função para escrever a matriz abre um arquivo, escreve no arquivo o tamanho da matriz e passa elemento por elemento os números da matriz para o arquivo de destino da escolha do usuário.

As funções de somar e multiplicar matrizes primeiro inicializam a matriz de destino usando a função de inicializar a matriz nula, e então os valores resultados da soma e multiplicação são passados para a terceira matriz que foi inicializada.

Já a função de transpor a matriz primeiro inicializa a matriz resultado usando a função de inicializar a matriz nula e então os elementos da matriz de entrada são passados para a matriz nula, mas com os índices invertidos.

3. Análise de complexidade

Apesar de haver outras funções que realizam operações constantes. Visto que há a aninhamento de dois laços que iteram por toda dimensão da matriz, a complexidade temporal das funções `criaMatriz`, `inicializaMatrizNula`, `escreverMatriz`, `multiplicarMatrizes`, `somarMatrizes` e `transpoeMatriz` é $\Theta(m*n)$. A complexidade espacial dessas funções também é $\Theta(m*n)$, visto que são usados vetores de ponteiros de linhas contíguas.

Já a função `destroiMatriz` possui funções do tipo `free()`, na qual o seu custo será ignorado para a avaliação da complexidade temporal da função. Visto que há somente operações unitárias, a complexidade assintótica de tempo é $O(1)$.

4. Estratégias de robustez

A robustez do programa foi implementada a partir do arquivo `'msgassert'` usando asserções aplicadas ao código considerando uma condição e uma mensagem de erro.

As condições que levam ao erro são:

Se não forem passados por parâmetro os arquivos das matrizes operando, da matriz em que são registrados os acessos à memória e da matriz em que é registrado o resultado ou se não for passado por parâmetro a operação a ser realizada.

Se a dimensões das matrizes forem impróprias para a operação.

Se a dimensão da matriz operando for inválida, ou seja, menor que zero.

Se o arquivo que for passado uma matriz é inválido.

5. Conclusão

Por fim, a solução adotada para a estrutura de dados foi criar vetor de ponteiros de linhas contíguas. Pode-se verificar que além de usar a mesma sintaxe do acesso que o método anterior (mesma que uma matriz estática), este método tem mais duas vantagens: somente precisa de duas operações de alocação de memória, e todos os elementos da matriz estão alocados sequencialmente na memória, o que facilita operações de cópia de matrizes (usando `"memcpy"`) ou de leitura/escrita da matriz para um arquivo (usando `"fread"` ou `"fwrite"`).

No trabalho, apesar da implementação da estrutura de dados ter sido desafiadora, a maior dificuldade foi para gerar os gráficos para a análise experimental, pelo fato do código do aplicativo usado para gerar os arquivos a serem plotados ter sido desenvolvido para um sistema Linux.

References

Chaimowicz, L. and Prates, R. (2020).

Slides virtuais da disciplina de estruturas de dados. Disponibilizado via moodle.

https://www.inf.ufpr.br/roberto/ci067/14_alocmat.html

6. Instruções de compilação e execução

O programa foi desenvolvido na linguagem C, compilada por GCC da GNU Compiler Collection no SO Linux.

Acesse o diretório TP. Compile os arquivos usando o comando make. O executável gerado estará em bin. Agora entre no diretório bin e execute o programa.

Para executar o programa deve se escolher alguma operação (-m, -s, -t), os arquivos que estão as matrizes (ex: -1 matriz1.txt -2 matriz2.txt), o arquivo com resultado (ex: -o resultado.txt), o arquivo onde será feito o registro de acesso (ex: -p registro.txt) e a opção '-l' indica se devem ser registrados os acessos à memória, ou apenas o tempo de execução.

